Agenda: More tricks

- Last time: DQN
  - Experience Replay
  - Fixed Q-targets.

- Today: More tricks
  - Double DQN
  - Prioritized Sweeping
  - Duelling Network

- Double DQN (Deep Reinforcement Learning with Double Q-Learning, Van Hasselt et al, AAAI 2016)
- Prioritized Replay (Prioritized Experience Replay, Schaul et al, ICLR 2016)
- Dueling DQN (best paper ICML 2016) (Dueling Network Architectures for Deep Reinforcement Learning, Wang et al, ICML 2016)

Double DQN

- Problem: Over-estimation of Q-Learning
- Solution: Double Q-Learning
- Extension to DQN: Double DQN

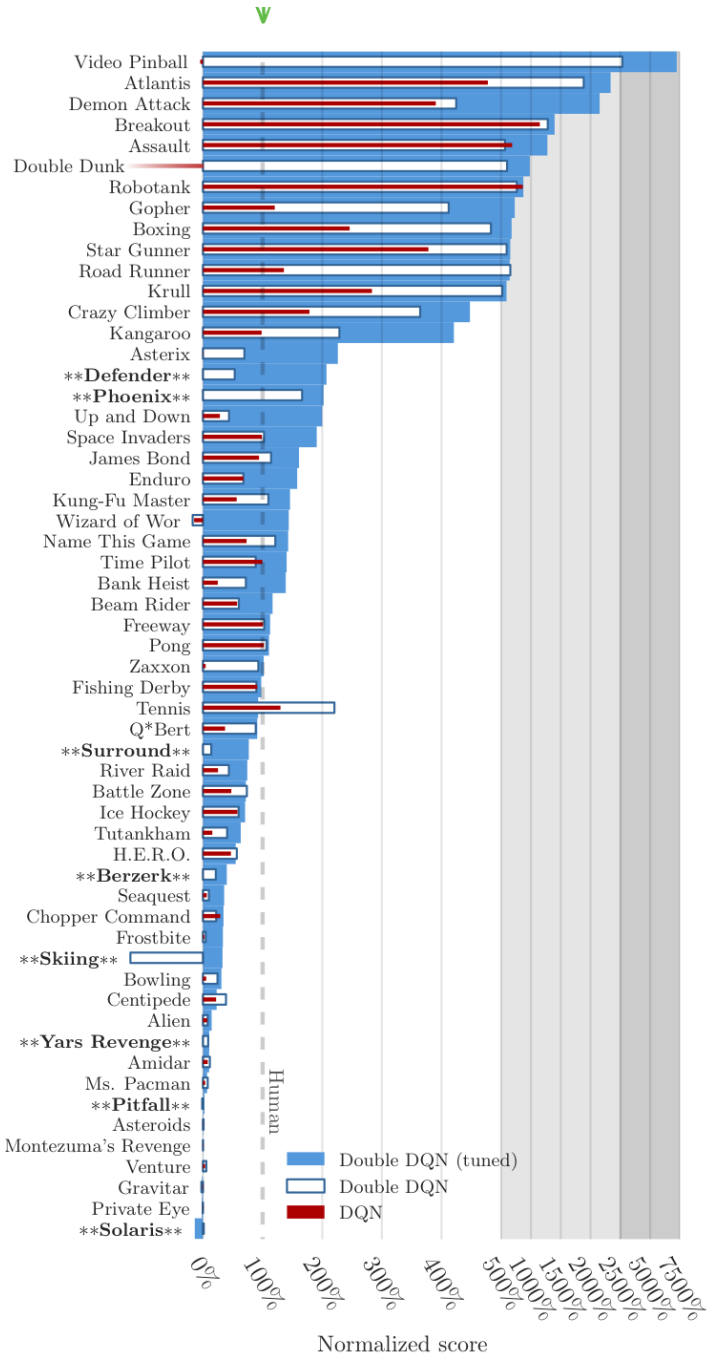


Figure 4: Normalized scores on 57 Atari games, tested for 100 episodes per game with human starts. Compared to Mnih et al. (2015), eight games additional games were tested. These are indicated with stars and a bold font.

- 49 + 8 = 57 games
- van Hasselt, Guez, Silver, 2015.

- Bias Problem with Q-Learning
  - $Q(s,a) \leftarrow R + \gamma \max_{a'} Q(s', a')$
  - Max of estimated State-Action Value is a biased estimate of the max

    over-estimate

    i.e

$$\mathbb{E}\left[\max_a \hat{Q}(s,a)\right] \geq \max_a Q(s,a)$$

Th^m: $\mathbb{E} \max_n X_n \geq \max_n \mathbb{E} X_n$

Proof: $\max_n X_n \geq \mathbb{E} X_n \left(\triangleq \int X_n(w)\, P(dw)\right)$

$\Rightarrow \mathbb{E} \max_n X_n \geq \mathbb{E} X_n$

$\Rightarrow \mathbb{E} \max_n X_n \geq \max_n \mathbb{E} X_n$ *

• Solution: Double Q-Learning.

1: Initialize $Q_1(s, a)$ and $Q_2(s, a), \forall s \in S, a \in A$ $t = 0$, initial state $s_t = s_0$
2: **loop**
3: Select a_t using ϵ-greedy $\pi(s) = \arg\max_a [Q_1(s_t, a) + Q_2(s_t, a)]$
4: Observe (r_t, s_{t+1})
5: **if** (with 0.5 probability) **then**
6:
$$Q_1(s_t, a_t) \leftarrow Q_1(s_t, a_t) + \alpha(r_t + Q_2(s_{t+1}, \arg\max_{a'} Q_1(s_{t+1}, a')) - Q_1(s_t, a_t))$$
7: **else**
8:
$$Q_2(s_t, a_t) \leftarrow Q_2(s_t, a_t) + \alpha(r_t + Q_1(s_{t+1}, \arg\max_{a'} Q_2(s_{t+1}, a')) - Q_2(s_t, a_t))$$

Evalute { Q_1 prob $\frac{1}{2}$; Q_2 prob $\frac{1}{2}$

9: **end if**
10: $t = t + 1$
11: **end loop**

---

i.e. decouple evaluation & selection

⇓

How ?

- Q-learning

Q shifting

Both depending on Q

Target

Approxi

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \left(\left[R_{t+1} + \gamma \max_{a'} Q(S_{t+1}, a') \right] - Q(S_t, A_t) \right)$$

$$Q\left(S_{t+1}, \operatorname*{argmax}_{a'} Q(S_{t+1}, a')\right)$$

Here : Select a' from Q Evalute Q

- Idea : Select a' from Q_1, Evaluate from Q_2

(Select a' from Q_2, Evaluate from Q_1)

- Double Q-learning:

$$Q_1(S_t, A_t) \leftarrow Q_1(S_t, A_t) + \alpha\left(\left[R_{t+1} + \gamma Q_2\left(S_{t+1}, \underset{a'}{\operatorname{argmax}}\, Q_1(S_{t+1}, a')\right)\right] - Q_1(S_t, A_t)\right)$$

evaluate action — select action

$$Q_2(S_t, A_t) \leftarrow Q_2(S_t, A_t) + \alpha\left(\left[R_{t+1} + \gamma Q_1\left(S_{t+1}, \underset{a'}{\operatorname{argmax}}\, Q_2(S_{t+1}, a')\right)\right] - Q_2(S_t, A_t)\right)$$

- Evaluation/Selection decoupled.

- Equivalently,

$$\Delta Q_1(S_t, A_t) \leftarrow \alpha \left(\left[R_{t+1} + \gamma Q_2\left(S_{t+1}, \underset{a'}{\operatorname{argmax}}\, Q_1(S_{t+1}, a')\right) \right] - Q_1(S_t, A_t) \right)$$

and

$$\Delta Q_2(S_t, A_t) \leftarrow \alpha \left(\left[R_{t+1} + \gamma Q_1\left(S_{t+1}, \underset{a'}{\operatorname{argmax}}\, Q_2(S_{t+1}, a')\right) \right] - Q_2(S_t, A_t) \right)$$

- Extension from 2×Q-learning → 2×DQN



Double DQN

$$\Delta w = \alpha \left[\left\{ R_{t+1} + \gamma \hat{Q}\left(S_{t+1}, \underset{a'}{\operatorname{argmax}}\, \hat{Q}(S_{t+1}, a', w), w^-\right) \right\} - \hat{Q}(S_t, A_t, w) \right] \times \nabla_w \hat{Q}(S_t, A_t, w)$$

action evaluation: w^-

action selection: w

- Current Q-network w:

  action selection.

- Older Q-network w^-:

  action evaluation.

} decoupled

DQN vs double DQN

$$\Delta w = \alpha \left[Y_t^{?} - Q(S_t, A_t, w_t) \right] Q(S_t, A_t, w_t)$$

- DQN:

$$Y_t^{DQN} = R_{t+1} + \gamma \max_a Q(S_{t+1}, a, w_t)$$

- Double DQN:

$$Y_t^{2\times DQN} = R_{t+1} + \gamma Q(S_{t+1}, \arg\max_a Q(S_{t+1}, a, w_t), w_t^-)$$

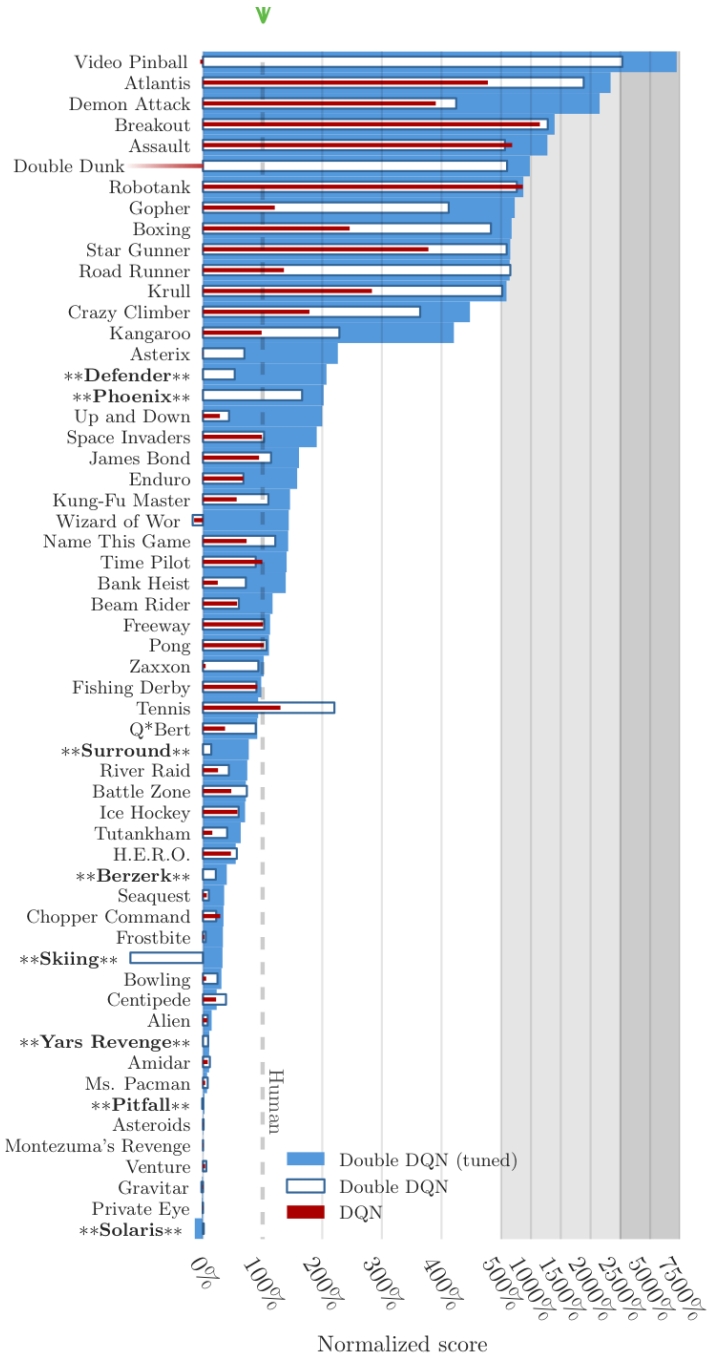


Figure 4: Normalized scores on 57 Atari games, tested for 100 episodes per game with human starts. Compared to Mnih et al. (2015), eight games additional games were tested. These are indicated with stars and a bold font.

- 49 + 8 = 57 games
- van Hasselt, Guez, Silver, 2015.

Summary on 2×DQN

- Q-learning: over-optimistic
- Overestimation: common & severe.
- 2×DQN: { reduce over-optimism; more stable/reliable }
- Easy transition from DQN
- 2×DQN: ⇒ { better policy; SOTA for Atari }.

Prioritized Replay

Prioritized Replay (Prioritized Experience Replay, Schaul et al, ICLR 2016)

Recall

DQNs: Experience Replay

- To help remove correlations, store dataset (called a **replay buffer**) $\mathcal{D}$ from prior experience

s_1, a_1, r_2, s_2
s_2, a_2, r_3, s_3
s_3, a_3, r_4, s_4
...
$s_t, a_t, r_{t+1}, s_{t+1}$

$\rightarrow \quad s, a, r, s'$

- To perform experience replay, repeat the following:
  - $(s, a, r, s') \sim \mathcal{D}$: sample an experience tuple from the dataset
  - Compute the target value for the sampled s: $r + \gamma \max_{a'} \hat{Q}(s', a'; \boldsymbol{w})$
  - Use stochastic gradient descent to update the network weights

$$\Delta \boldsymbol{w} = \alpha(r + \gamma \max_{a'} \hat{Q}(s', a'; \boldsymbol{w}) - \hat{Q}(s, a; \boldsymbol{w}))\nabla_{\boldsymbol{w}} \hat{Q}(s, a; \boldsymbol{w})$$

- Experience Replay : critical in DQN.
- Q :
  - ① Are all experiences equally important?
  - ② If not, how to better use them ?
- A :
  - ①. NO !
  - ②. Replace uniform sampling

    Give more weight to "important" experiences ?

    "how to define?"

B EXPERIMENTAL DETAILS

B.1 BLIND CLIFFWALK

For the Blind Cliffwalk experiments (Section 3.1 and following), we use a straight-forward Q-learning (Watkins & Dayan, 1992) setup. The Q-values are represented using either a tabular look-up table, or a linear function approximator, in both cases represented $Q(s,a) := \theta^\top \phi(s,a)$. For each transition, we compute its TD-error using:

$$\delta_t \quad := \quad R_t + \gamma_t \max_a Q(S_t, a) - Q(S_{t-1}, A_{t-1}) \tag{2}$$

and update the parameters using stochastic gradient ascent:

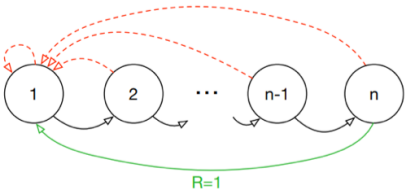
$$\theta \quad \leftarrow \quad \theta + \eta \cdot \delta_t \cdot \nabla_\theta Q\big|_{S_{t-1}, A_{t-1}} = \theta + \eta \cdot \delta_t \cdot \phi(S_{t-1}, A_{t-1}) \tag{3}$$

For the linear FA case we use a very simple encoding of state as a 1-hot vector (as for tabular), but concatenated with a constant bias feature of value 1. To make generalization across actions impossible, we alternate which action is 'right' and which one is 'wrong' for each state. All elements are initialized with small values near zero, $\theta_i \sim \mathcal{N}(0, 0.1)$.

We vary the size of the problem (number of states n) from 2 to 16. The discount factor is set to $\gamma = 1 - \frac{1}{n}$ which keeps values on approximately the same scale independently of n. This allows us to used a fixed step-size of $\eta = \frac{1}{4}$ in all experiments.

The replay memory is filled by exhaustively executing all 2^n possible sequences of actions until termination (in random order). This guarantees that exactly one sequence will succeed and hit the final reward, and all others will fail with zero reward. The replay memory contains all the relevant experience (the total number of transitions is $2^{n+1} - 2$), at the frequency that it would be encountered when acting online with a random behavior policy. Given this, we can in principle learn until convergence by just increasing the amount of computation; here, convergence is defined as a mean-squared error (MSE) between the Q-value estimates and the ground truth below 10^{-3}.

Potential Impact of Ordering Episodic Replay Updates



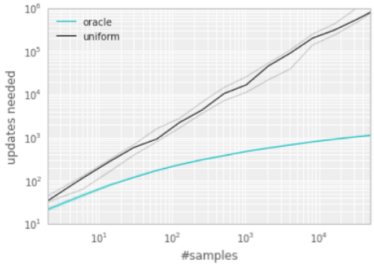


Figure: Schaul, Quan, Antonoglou, Silver ICLR 2016

- Schaul, Quan, Antonoglou, Silver ICLR 2016
- Oracle: picks (s, a, r, s') tuple to replay that will minimize global loss
- Exponential improvement in convergence
  - Number of updates needed to converge
- Oracle is not a practical method but illustrates impact of ordering

- Order of replay MATTERS!

How to order / prioritize ?

- Given an experience $\{S_t, A_t, R_{t+1}, S_{t+1}\}$
- Define priority p_i for each experience

$$p_i = |\delta_i| + \varepsilon$$

where $\delta_t \triangleq \left\{R_{t+1} + \gamma \max_{a'} Q(S_{t+1}, a', w^-)\right\} - Q(S_t, A_t, w)$

Intuition

- δ_t should contain much info about transitions
- { the larger $|\delta_t|$ is, the more informative,

- Assign sampling probability

$$P(i) = \frac{p_i^\alpha}{\sum_j p_j^\alpha} = \frac{(|\delta_i| + \varepsilon)^\alpha}{\sum_j (|\delta_j| + \varepsilon)^\alpha}.$$

$(\alpha \geq 0, \quad \varepsilon \geq 0)$

- Stochastic Prioritization

- Special cases are given below.

Algorithm 1: Highest Priovitization (Maximum TD error)

- Update TD error δ_i every update
- Pick the largest $|\delta_i|$

$$t_* = \underset{1 \le i \le N}{\operatorname{argmax}} |\delta_i|$$

- Update t_*^{th} tuple $\{S_{t_*}, A_{t_*}, R_{t_*+1}, S_{t_*+1}\}$.

- This corresponds to $\alpha = \infty$.

Algorithm 2: Proportional

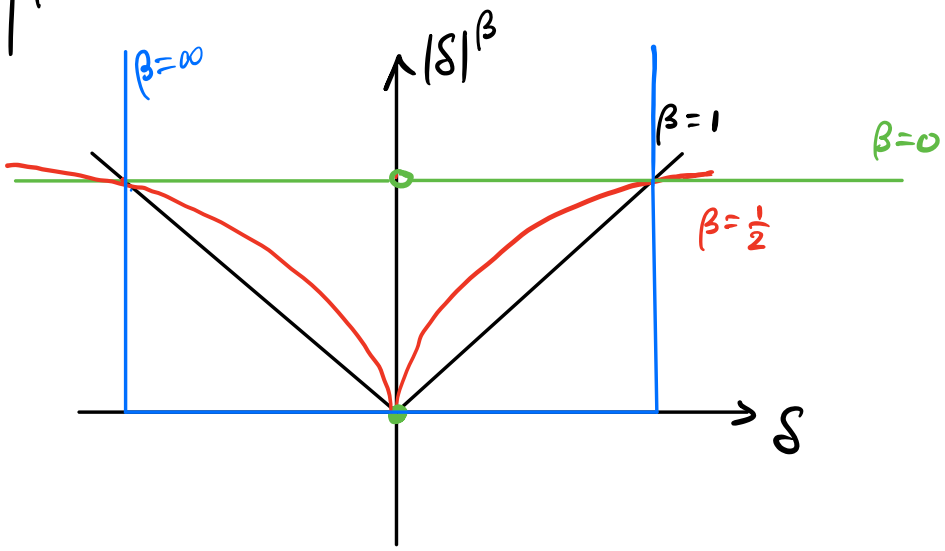
(Stochastic Prioritization)

- Update TD error δ_i (online / batch)
- Sample tuples

$$p_i = \frac{\left(|\delta_i| + \varepsilon\right)^{\alpha}}{\sum_j \left(|\delta_j| + \varepsilon\right)^{\alpha}}$$

Remarks

- $|\delta|^\beta$



- $$\delta = \begin{cases} 0, & \text{uniform sampling} \\ 1, & \text{proportional sampling} \\ \infty, & \text{maximum sampling} \end{cases}$$

- $\varepsilon > 0 \Rightarrow$ even if $\delta_i = 0$, keep sampling

- Typically. $\beta \in [0, 1]$.

Algorithm 3: Rank-based

- Define

$$p_i = \frac{1}{\text{rank}\{|\delta_i|\}}$$

- More robust / stable.

Large $\delta_i \Rightarrow$ too much update.

Algorithm 4: Asymmetric Prioritization

Q: Is +ve δ_i more important than -ve δ_i ?

If it is, place more weights to +ve δ_i

Annealling the Bias

- MC is unbiased

$$E_{Unif}\,\hat{Q} = Q$$

- Stochastic Prioritity is Biased!

$$E_{p}\,\hat{Q} \neq Q \,.$$

- Change of measure (Importance Sampling, IS)

$$E_p\left(\frac{\text{Unif}}{p}\hat{Q}\right) = Q$$

- More generally, use weight

$$w_i \propto \left(\frac{1}{p(i)}\right)^{\beta}$$

where $\beta = 1 \iff$ IS.

that they only scale the update downwards.

In typical reinforcement learning scenarios, the unbiased nature of the updates is most important near convergence at the end of training, as the process is highly non-stationary anyway, due to changing policies, state distributions and bootstrap targets; we hypothesize that a small bias can be ignored in this context (see also Figure 12 in the appendix for a case study of full IS correction on Atari). We therefore exploit the flexibility of *annealing* the amount of importance-sampling correction over time, by defining a schedule on the exponent β that reaches 1 only at the end of learning. In practice, we linearly anneal β from its initial value β_0 to 1. Note that the choice of this hyperparameter interacts with choice of prioritization exponent α; increasing both simultaneously prioritizes sampling more aggressively at the same time as correcting for it more strongly.

Algorithm 1 Double DQN with proportional prioritization

Input: minibatch k, step-size η, replay period K and size N, exponents α and β, budget T.
Initialize replay memory $\mathcal{H} = \emptyset$, $\Delta = 0$, $p_1 = 1$
Observe S_0 and choose $A_0 \sim \pi_\theta(S_0)$
for $t = 1$ **to** T **do**
  Observe S_t, R_t, γ_t
  Store transition $(S_{t-1}, A_{t-1}, R_t, \gamma_t, S_t)$ in $\mathcal{H}$ with maximal priority $p_t = \max_{i<t} p_i$
  if $t \equiv 0 \mod K$ **then**
    for $j = 1$ **to** k **do**
      Sample transition $j \sim P(j) = p_j^\alpha / \sum_i p_i^\alpha$
      Compute importance-sampling weight $w_j = (N \cdot P(j))^{-\beta} / \max_i w_i$
      Compute TD-error $\delta_j = R_j + \gamma_j Q_{\text{target}}(S_j, \arg\max_a Q(S_j, a)) - Q(S_{j-1}, A_{j-1})$
      Update transition priority $p_j \leftarrow |\delta_j|$
      Accumulate weight-change $\Delta \leftarrow \Delta + w_j \cdot \delta_j \cdot \nabla_\theta Q(S_{j-1}, A_{j-1})$
    end for
    Update weights $\theta \leftarrow \theta + \eta \cdot \Delta$, reset $\Delta = 0$
    From time to time copy weights into target network $\theta_{\text{target}} \leftarrow \theta$
  end if
  Choose action $A_t \sim \pi_\theta(S_t)$
end for

e.g. Blind Cliff

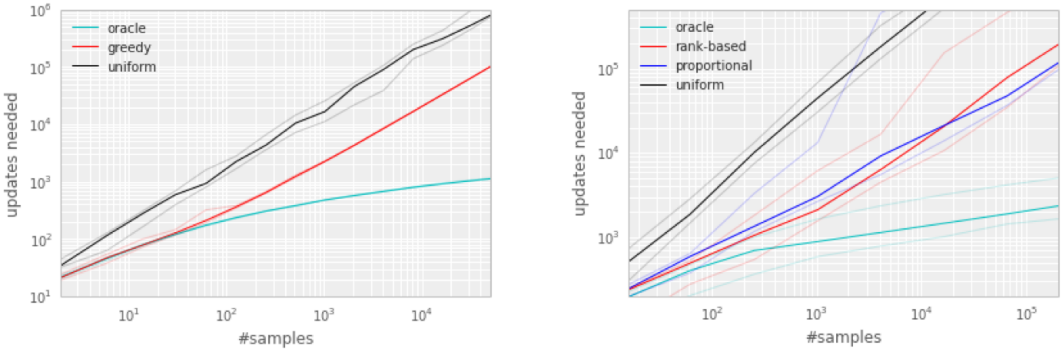


Figure 2: Median number of updates required for Q-learning to learn the value function on the Blind Cliffwalk example, as a function of the total number of transitions (only a single one of which was successful and saw the non-zero reward). Faint lines are min/max values from 10 random initializations. Black is uniform random replay, cyan uses the hindsight-oracle to select transitions, red and blue use prioritized replay (rank-based and proportional respectively). The results differ by multiple orders of magnitude, thus the need for a log-log plot. In both subplots it is evident that replaying experience in the right order makes an enormous difference to the number of updates required. See Appendix B.1 for details. **Left:** Tabular representation, greedy prioritization. **Right**: Linear function approximation, both variants of stochastic prioritization.

Performance of Prioritized Replay vs Double DQN

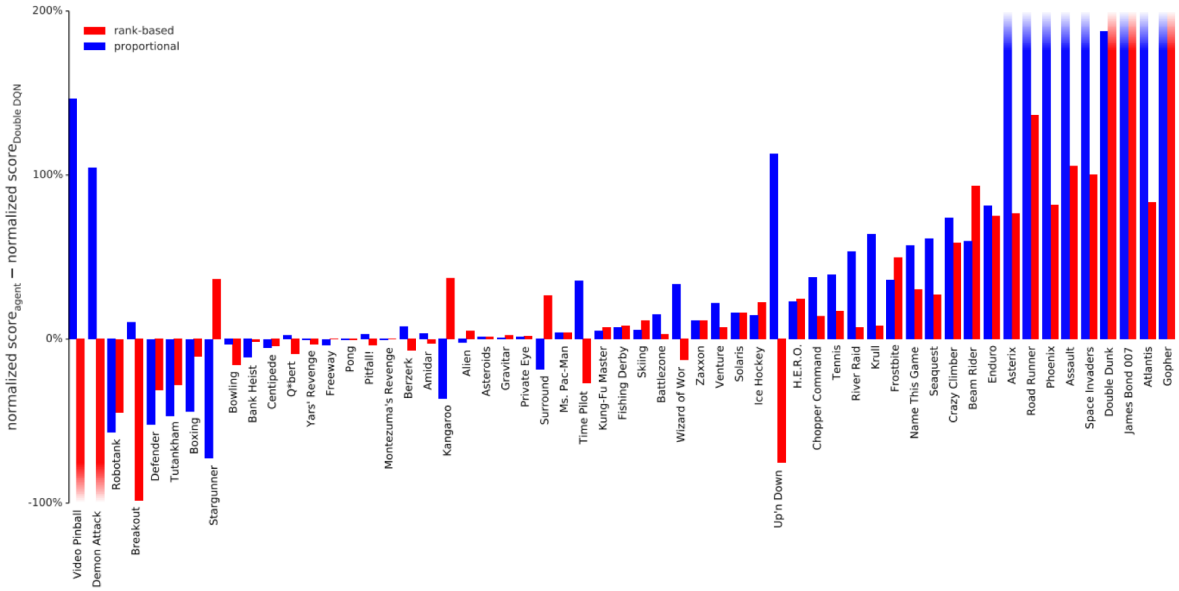


Figure: Schaul, Quan, Antonoglou, Silver ICLR 2016

Learning Speed

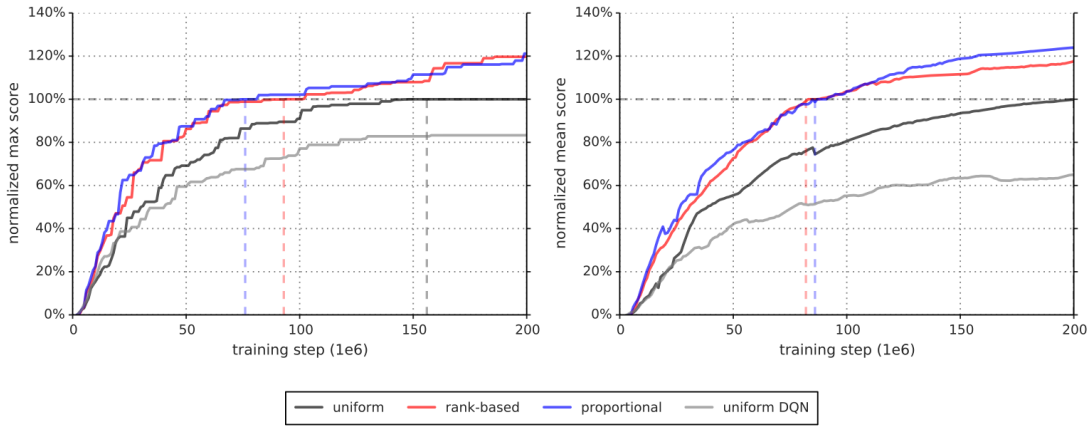


Figure 4: Summary plots of learning speed. **Left:** median over 57 games of the maximum baseline-normalized score achieved so far. The baseline-normalized score is calculated as in Equation 4 but using the maximum Double DQN score seen across training is used instead of the human score. The equivalence points are highlighted with dashed lines; those are the steps at which the curves reach 100%, (i.e., when the algorithm performs equivalently to Double DQN in terms of median over games). For rank-based and proportional prioritization these are at 47% and 38% of total training time. **Right:** Similar to the left, but using the mean instead of maximum, which captures cumulative performance rather than peak performance. Here rank-based and proportional prioritization reach the equivalence points at 41% and 43% of total training time, respectively. For the detailed learning curves that these plots summarize, see Figure 7.

Dueling DQN

- Dueling DQN (best paper ICML 2016) (Dueling Network Architectures for Deep Reinforcement Learning, Wang et al, ICML 2016)

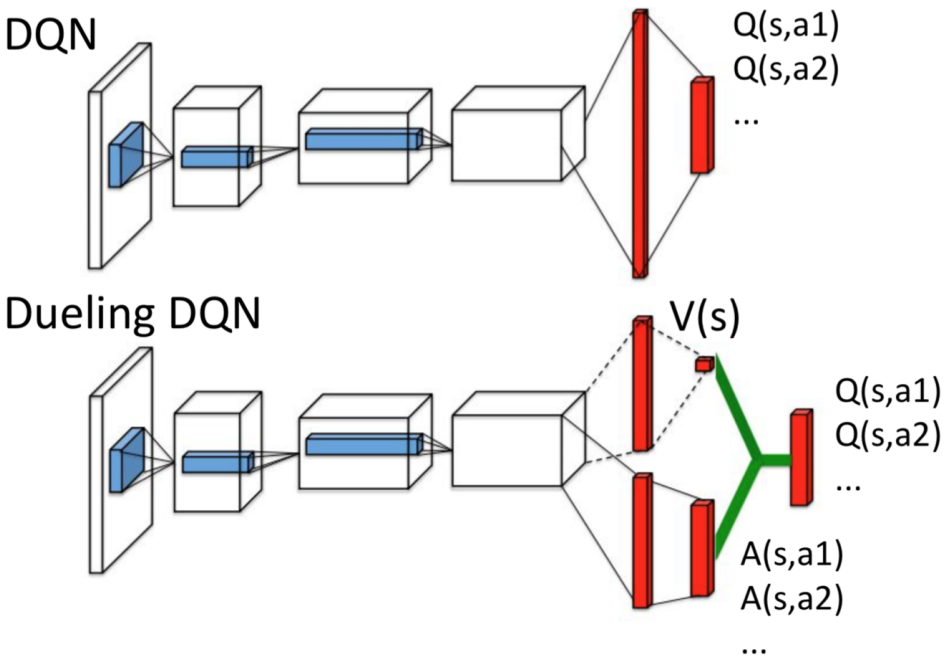
Value & Advantage Function

- Intuition: Features needed to accurately represent value may be different than those needed to specify difference in actions
- E.g.
  - Game score may help accurately predict $V(s)$
  - But not necessarily in indicating relative action values $Q(s, a_1)$ vs $Q(s, a_2)$
- Advantage function (Baird 1993)

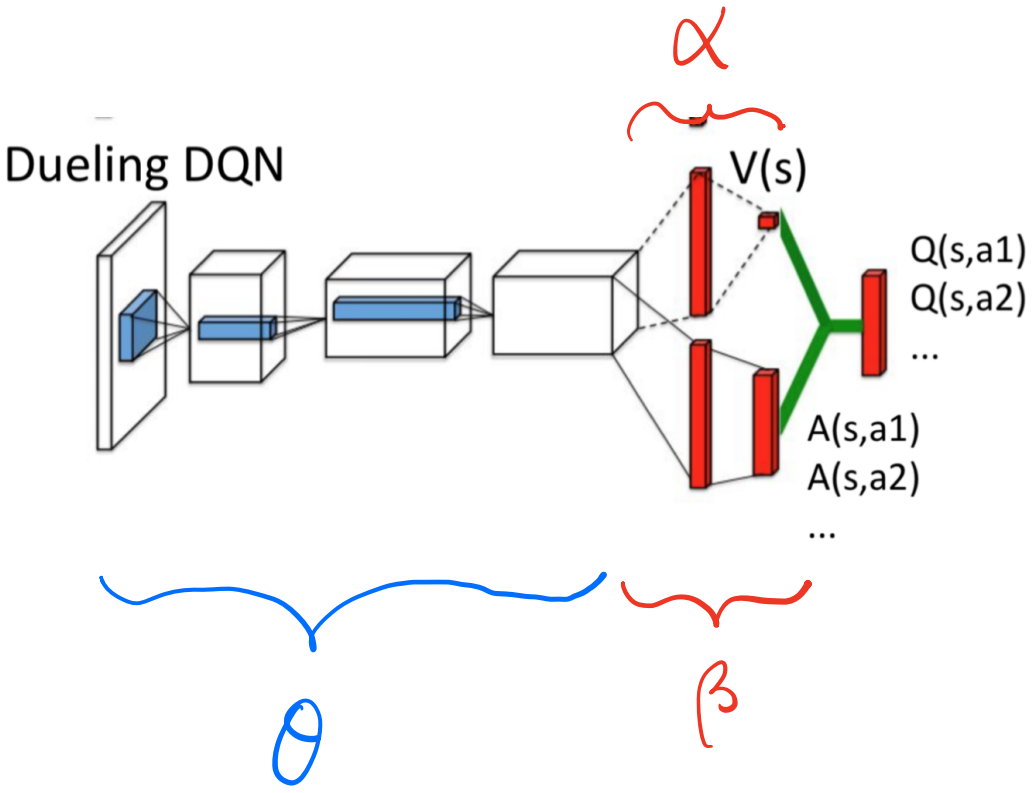
$$A^{\pi}(s, a) = Q^{\pi}(s, a) - V^{\pi}(s)$$

$$= Q^{\pi}(s, a) - Q^{\pi}(s, \pi(a))$$

Dueling DQN



Wang et.al., ICML, 2016



- $Q^{\pi}(s,a) = V^{\pi}(s) + A^{\pi}(s,a)$

- Q, V are estimated in one go.

(Killing 2 birds with 1 stone, 一箭双雕)

Identifiability

- Non-identifiable.

$$Q^{\pi}(s,a) = V^{\pi}(s) + A^{\pi}(s,a)$$

$$= \left[V^{\pi}(s) + C(s)\right] + \left[A^{\pi}(s,a) - C(s)\right]$$

$$\forall\, C(s)$$

- Need one constraint.

- **Solution 1:**

$$Q^{\pi}(s,a) = V^{\pi}(s) + \left[A^{\pi}(s,a) - \max_{a'} A^{\pi}(s,a') \right]$$

i.e. Force $Q^{\pi}(s,a) = V(s)$ for

$$a = \arg\max_{a'} A^{\pi}(s,a),$$

the best action suggested by $A^{\pi}(s,a)$.

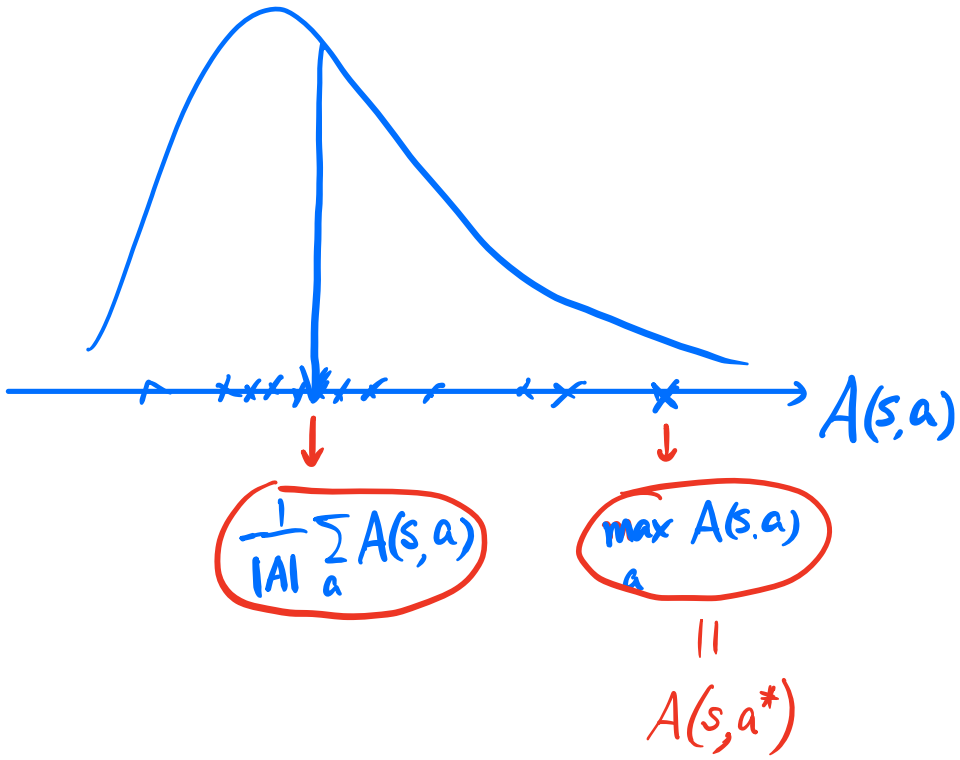
- **Solution 2:**

$$Q^{\pi}(s,a) = V^{\pi}(s) + \left[A^{\pi}(s,a) - \frac{1}{|A(s)|} \sum_{a' \in A(s)} A^{\pi}(s,a') \right]$$

i.e. use mean as the baseline.

Q: Averages more stable than maximum?

A: Yes.



e.g. Corridor environment

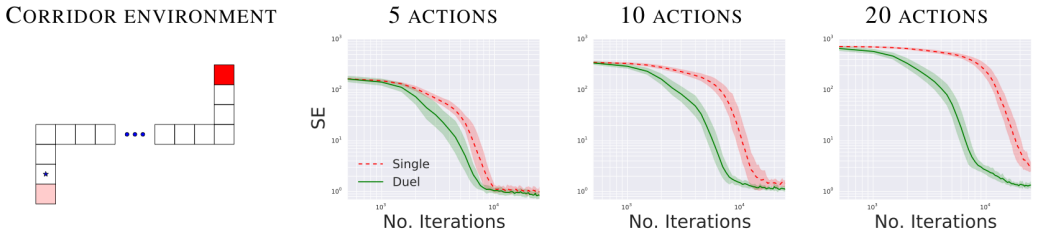


Figure 3. **(a)** The corridor environment. The star marks the starting state. The redness of a state signifies the reward the agent receives upon arrival. The game terminates upon reaching either reward state. The agent's actions are going up, down, left, right and no action. Plots **(b), (c)** and **(d)** shows squared error for policy evaluation with 5, 10, and 20 actions on a log-log scale. The dueling network (Duel) consistently outperforms a conventional single-stream network (Single), with the performance gap increasing with the number of actions.

Atari

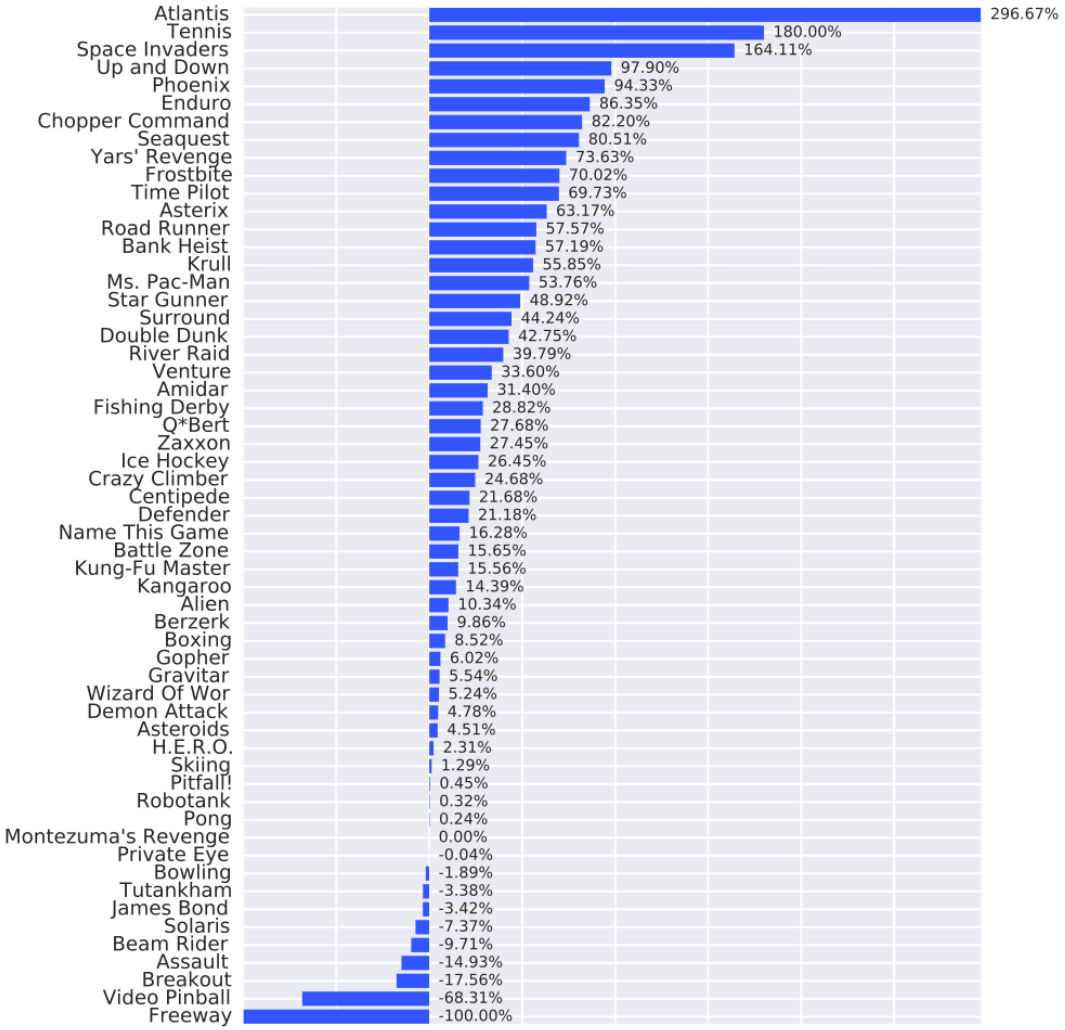


Figure 4. Improvements of dueling architecture over the baseline Single network of van Hasselt et al. (2015), using the metric described in Equation (10). Bars to the right indicate by how much the dueling network outperforms the single-stream network.

Atari

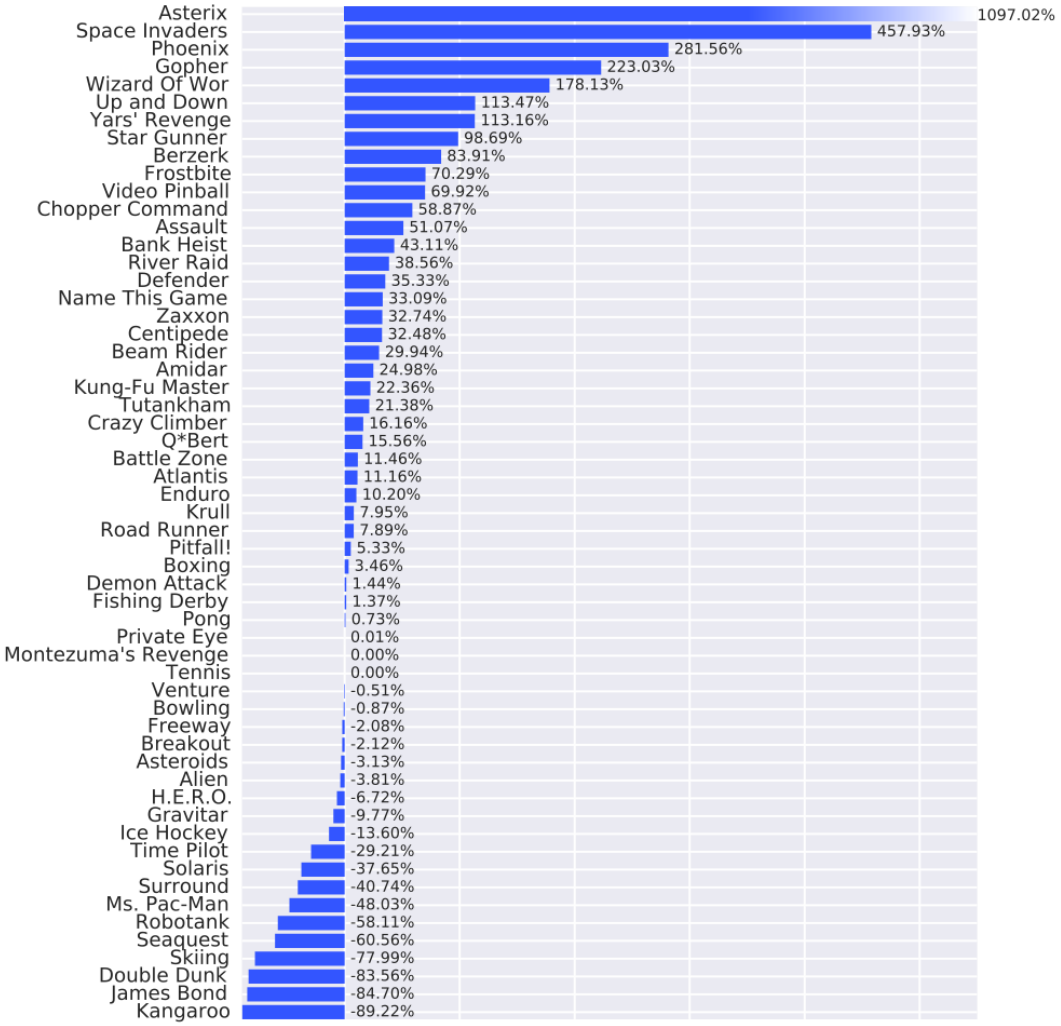


Figure 5. Improvements of dueling architecture over Prioritized DDQN baseline, using the same metric as Figure 4. Again, the dueling architecture leads to significant improvements over the single-stream baseline on the majority of games.

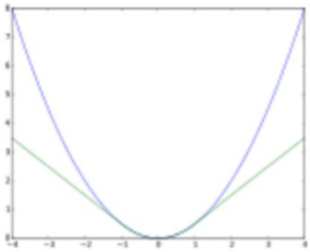
Summary

- DQN
  - Experience Raplay.
  - Fixed Q-target
- DQN+
  - Double DQN
  - Prioritized Replay
  - Dueling
  - ……

Practical Tips for DQN on Atari (from J. Schulman)

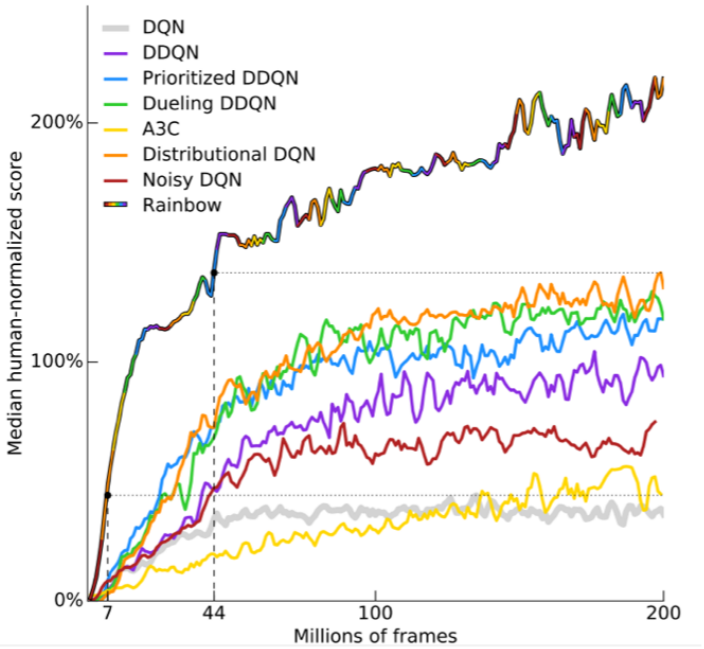
- DQN is more reliable on some Atari tasks than others. Pong is a reliable task: if it doesn't achieve good scores, something is wrong
- Large replay buffers improve robustness of DQN, and memory efficiency is key
  - Use uint8 images, don't duplicate data
- Be patient. DQN converges slowly—for ATARI it's often necessary to wait for 10-40M frames (couple of hours to a day of training on GPU) to see results significantly better than random policy
- Try Huber loss on Bellman error

$$L(x) = \begin{cases} \frac{x^2}{2} & \text{if } |x| \leq \delta \\ \delta|x| - \frac{\delta^2}{2} & \text{otherwise} \end{cases}$$



- Consider trying Double DQN—significant improvement from small code change
- To test out your data pre-processing, try your own skills at navigating the environment based on processed frames
- Always run at least two different seeds when experimenting
- Learning rate scheduling is beneficial. Try high learning rates in initial exploration period
- Try non-standard exploration schedules

Deep Reinforcement Learning 2018



- Hessel, Matteo, et al. "Rainbow: Combining Improvements in Deep Reinforcement Learning."
- Very active area of research!